Nova: Complete Project Report

Plan-first blueprint for the Nova desk assistant. Version 1.0, 22 Oct 2026. Owner: Devesh. Status: planning only, no new code yet.

0. How to use this report

- This is the single source of truth. We write everything here first, then build by following the phases in section 13 in order.
- The rules in section 12 apply from day one.
- Anything not yet decided is listed in section 16 with a default. If a decision changes, update this report before touching code.
- Working agreement: planning stays planning. Coding for a phase starts only when you say 'go ahead with code'.

1. Short introduction

Nova is a voice-controlled desk assistant for one user (you) on one Linux Mint PC. A small ESP32 with an OLED sits on the desk as Nova's face. You say 'Hey Nova' and Nova:

- listens and understands English speech, processed locally on the laptop;
- thinks with a cloud LLM, and answers in short spoken sentences;
- does tasks on your PC through a small set of approved, safe tools (open apps and sites, find files, run a project's tests, and more);
- shows what it is doing on the OLED (listening, thinking, working and so on) and in detail on a website;
- keeps your timer and to-do list visible, as it already does today;
- later, remembers your ideas and projects in an Obsidian vault.

What it is not: a product, a cloud service, or an unrestricted remote control. It only does what the approved tools allow, inside approved folders.

Headline demo: 'Hey Nova, open my project and run its tests.' Nova understands, opens the project, runs its tests, speaks the result, shows its state on the OLED, and logs everything on the website.

2. Goals, scope and definition of done

Goals

- A working, reliable personal assistant, ideally within about a month, with flexible hours.
- A safe design: Nova can only do what is registered and allowed.
- A project you can explain end to end: every part planned, tested and documented.

In scope for v1: wake word, push-to-talk, local speech-to-text, cloud LLM with tool calling, local text-to-speech, the tools in section 3B, the safety tiers, eight OLED states, website chat and activity log, the existing timer and to-dos.

Later (see phases 7 and 8): Obsidian memory, interruption while speaking, screen awareness, interactive browser automation, extra hardware, other users, a local LLM option.

Non-goals: custom PCB or enclosure, running AI models on the ESP32, unrestricted control of the PC, always-on screen recording, deleting or installing software.

Done when

1. At least 90% of a fixed scripted set of 30 commands work (27 or more).
2. The first spoken response starts in under 3 seconds where feasible (end of your speech to first audio).
3. The safety model is documented and its tests pass.
4. The headline demo runs end to end, three times in a row.

3. Function catalogue

Each function has an ID and the phase that delivers it. 'Existing' means already built and working.

3A. Voice and conversation

- **V1** Wake word 'Hey Nova' (phase 2 test, phase 4 build).
- **V2** Push-to-talk from the website, recording from the laptop microphone (phase 4).
- **V3** End-of-speech detection, so you can talk naturally and pause (phase 4).
- **V4** Local speech-to-text, English (phase 4).
- **V5** LLM conversation with short-term context (phase 3).
- **V6** Local text-to-speech, calm female voice, normal speed (phase 4).

- **V7** Microphone muted while Nova speaks, so it cannot hear itself (phase 4).
- **V8** Mic-off switch on the website and the voice command 'stop listening' (phase 4).
- **V9** Quick spoken acknowledgement for long tasks, such as 'On it.' (phase 4).
- **V10** Plain spoken explanation when something fails or is refused (phase 5).

3B. Computer control (tools)

- **C1** Open an approved app: browser, file manager, VS Code, text editor.
- **C2** Open a website (http or https only).
- **C3** Web search: opens the browser on a search page.
- **C4** Switch browser tabs.
- **C5** Open an approved folder.
- **C6** Find files inside approved folders.
- **C7** Type text into the text editor (with window verification).
- **C8** Run a registered project's test command and report pass or fail.
- **C9** Read a stored result aloud.
- **C10** Check the USB drive is mounted before any project work, and say so if it is not. All delivered in phase 5, with mock versions in phase 3.

3C. Productivity

- **P1** To-do list on the website and OLED (existing).
- **P2** Timer with focus layout and progress bar on the OLED (existing).
- **P3** Peek view of the next task on the OLED (existing).
- **P4** Proposed addition: voice control of to-dos and timer (add, complete, start, stop, check). These are cheap because the backend functions already exist (phase 5, confirm in section 16).

3D. Agent and safety

- **S1** Tool registry: every tool has a name, a schema for its arguments, and a risk tier.
- **S2** Four risk tiers and the confirmation flow.
- **S3** Allowed-folder guard with real-path checking.
- **S4** Confirmation by voice and on the website.
- **S5** Step limit of 5, verification after each step, stop and report on failure.
- **S6** Activity log of every state change, tool call, result and error.

- **S7** Forbidden list: delete, install, sudo, arbitrary shell, anything outside allowed folders.
- **S8** Tool output treated as data, never as instructions.
- **S9** Microphone privacy: visible listening state, mic-off switch, no audio stored.

3E. Device (ESP32 and OLED)

- **D1** Animated eyes: blink, wink, look around (existing).
- **D2** Eight Nova states: idle, listening, thinking, speaking, working, saved, error, disconnected.
- **D3** Receive state from the laptop over HTTP.
- **D4** Heartbeat, and the disconnected state when the laptop goes quiet.
- **D5** Short text and step display (for example 'Working 2/4').
- **D6** Timer and to-do displays continue to work, even offline (existing).

3F. Website

- **W1** To-dos and timer (existing).
- **W2** Chat panel (typed requests use the same pipeline as voice).
- **W3** Activity log with step details and full outputs.
- **W4** Confirmation prompts.
- **W5** Push-to-talk button.
- **W6** Status and mic switch.

3G. Memory (phase 7)

- **M1** Capture an idea by voice into the Ideas sub-brain.
- **M2** Recall ideas and project status by voice.
- **M3** Project notes, one folder per project.
- **M4** Index at the root of the main brain, used to route requests.
- **M5** Nova proposes updates and you confirm important ones.
- **M6** Inspect, correct and delete memory.

3H. System

- **Y1** Config files. **Y2** Logging. **Y3** Health checks. **Y4** One command to start everything. **Y5** Swappable providers for speech, LLM, wake word and device.

4. How it works, end to end

4.1 What already exists

- **Firmware:** eyes with blink, wink and look-around; Wi-Fi with a background polling task on core 0; a local timer state machine; the peek view; the focus layout with countdown and progress bar.
- **Server:** a FastAPI app with to-dos, timer, a compact state endpoint for the ESP32, and the phone page. To-dos are saved in a small JSON file. The ESP32 polls every 2 seconds.

4.2 Big picture

- **Phone or laptop browser** shows the website.
- **Laptop** runs two programs: the Nova core (website, LLM, tools, safety, device push) and the voice worker (microphone, wake word, speech-to-text, speech output).
- **ESP32 and OLED** is the face and runs a small HTTP server to receive states.
- **Cloud LLM** reasons and picks tools. It never executes anything itself.

4.3 A request, step by step

1. The voice worker hears 'Hey Nova' and reports listening.
2. It records until you pause, transcribes locally, and sends the text to the core.
3. The core sends the text, the tool list and Nova's instructions to the LLM.
4. The LLM returns a tool call or a reply. The core checks every tool call against the registry, the tier and the allowed folders.
5. Allowed tools run one at a time. Each result is verified, logged and fed back to the LLM, up to 5 steps.
6. The final short reply is spoken. The mic stays muted until it ends.
7. Every state change is pushed to the ESP32 and written to the activity log.

4.4 Latency budget (targets to be measured in phase 2)

- Recording ends after a short silence (about 0.7 to 1 s).
- Speech-to-text: 1 s or less for a short command.
- LLM first sentence: about 1.2 s.
- Text-to-speech first audio: about 0.5 s.
- Total: under 3 s for short replies. Long tasks get a quick acknowledgement first.

- The laptop has 2 cores, so phase 2 decides whether these hold. Fallbacks: a smaller model, a cloud speech-to-text option, or acknowledgement-first.

5. Hardware

5.1 Inventory and roles

- **ESP32 Dev Module** (ESP32-D0WD-V3, revision 3.0, dual core 240 MHz, Wi-Fi and Bluetooth): Nova's dedicated controller.
- **0.96 inch OLED**, I2C, 128 by 64, SSD1306, address 0x3C: Nova's face.
- **USB cable:** must carry data, not just power. Earlier upload errors suggest using a short, good cable.
- **Laptop:** Lenovo IdeaPad S145-15IIL, Intel Core i3-1005G1 (2 cores), 12 GB RAM, integrated graphics, Linux Mint with Xfce on X11.
- **Audio:** headphones with a microphone for development (prevents echo). The laptop microphone and speakers are the fallback.
- **Router:** home Wi-Fi on 2.4 GHz (the ESP32 does not support 5 GHz).
- **USB pen drive:** holds your projects.
- **No new hardware is needed for v1.**

5.2 Wiring

- OLED VCC to ESP32 3V3.
- OLED GND to ESP32 GND.
- OLED SDA to GPIO 21.
- OLED SCL to GPIO 22.
- I2C clock 800 kHz, fall back to 400 kHz if the screen glitches. No reset pin.

5.3 Network

- ESP32 and laptop on the same 2.4 GHz network.
- Reserve fixed IP addresses for both in the router (or use mDNS names).
- Ports: the Nova core on 8000, the ESP32 server on 80.

5.4 Optional later hardware (not needed now)

- A push button as a push-to-talk and mic-off switch.

- A spare ESP32.
- A USB microphone and speaker.
- A small case.

5.5 Hardware rules

- If an upload fails with serial noise, lower the upload speed to 115200, hold the BOOT button while connecting, or change the cable or port.
- Keep GPIO 2, 12 and 15 free of external connections (boot pins).
- Power the OLED from 3V3 only.

6. Software stack

- **Firmware:** Arduino IDE with the ESP32 board package, board 'ESP32 Dev Module'. Libraries: Adafruit SSD1306, Adafruit GFX, ArduinoJson 7, plus the ESP32 core's built-in WiFi, HTTPClient and WebServer.
- **Laptop core:** Python (the installed version, confirmed in phase 0), FastAPI, uvicorn, pydantic.
- **Voice:** Picovoice Porcupine for the wake word, a voice-activity detector (Silero VAD or webrtcvad), faster-whisper with tiny.en or base.en, Piper for text-to-speech (Kokoro as an upgrade), sounddevice for audio.
- **LLM:** Grok through the xAI API as the first candidate, behind a swappable interface. Access and cost are verified in phase 0.
- **Automation:** Python subprocess without a shell, plus xdotool and wmctrl on X11.
- **Website:** the existing page extended with chat, activity log and confirmations. No heavy framework.
- **Data:** to-dos stay in the existing JSON file. The activity log is a small SQLite database. Config in TOML files, secrets in an ignored .env file.
- **Testing and tools:** pytest, a scripted command runner, Git, curl.
- **Later:** Obsidian desktop app and Markdown files (phase 7).
- **Version rule:** pin versions in requirements at the end of phase 0, so a fresh setup behaves the same.

7. Firmware design

- **Existing modules stay:** eyes and rendering, the network polling task, the timer state machine, the views (peek and focus).
- **New:** a Nova state module and a small HTTP server running in its own task on core 0, next to the polling task. The handler only writes the received state into a shared structure guarded by a mutex. The display loop on core 1 only reads it, so the eyes never wait on the network.
- **State rules:** the latest state from the laptop wins until its time-to-live ends or a new state arrives. Disconnected is decided locally: no heartbeat for 15 seconds. Nova states temporarily override the peek and focus views, then the device returns to whatever it was showing.
- **Display rules:** plain ASCII only, text up to 40 characters, error messages up to about 20 characters, no blocking waits, frame-rate-independent animation, redraw only when something changed.
- **State looks:** idle (normal eyes), listening (wide attentive eyes and a marker), thinking (eyes looking aside and a small activity indicator), speaking (small talking indicator), working (compact layout with 'Working n/N'), saved (happy eyes and a check for about 2 s), error (short message), disconnected (sleepy eyes and a dot, local timer keeps running).
- **Security rule:** the display endpoint accepts requests only with the shared token (or from the laptop's address).

8. Laptop software design

- **Processes:** Nova core and voice worker, talking over local HTTP. Audio never blocks the web server or device updates.
- **Core modules:** API, website files, LLM provider, tool registry, safety guard, executor, agent loop, activity log, device client, config loader, project registry, and the existing to-do and timer code.
- **Voice worker modules:** audio input, wake word, end-of-speech detection, speech-to-text, text-to-speech, mic mute control.
- **Config files:** `nova.toml` (ports, device address, allowed folders, provider choices, timeouts), `projects.toml` (project name, folder, test command, timeout), `.env.example` (key names only). Real secrets are never committed.

- **Conversation context:** the last few turns, kept in memory and cleared after an idle period. Durable memory waits for phase 7.
- **LLM instructions (summary):** concise and friendly, short spoken answers, long details go to the website; use only registered tools; never claim a result that was not verified; say 'unconfirmed' when unsure; ask when a request is ambiguous; refuse forbidden actions plainly.

9. API contracts (draft)

Nova core, new

- `POST /api/chat` with the text; returns the reply and an activity id.
- `POST /api/voice/ptt/start` and `/stop`: website push-to-talk.
- `GET /api/activity` since a time: entries with timestamp, kind (state, tool_call, result, error), tier, summary, details.
- `GET /api/confirmations` and `POST /api/confirm/{id}` with approve true or false.
- `GET /api/status`: Nova state, device connected, mic on or off.
- `POST /api/mic` with enabled true or false.

Nova core, unchanged: the existing to-do, timer and compact state endpoints.

ESP32: `POST /display` with state, optional short ASCII text, optional step and steps, optional time-to-live; and `GET /health`.

Rules for contracts: any endpoint that can cause an action or approve one requires the access token. Errors return a clear code and message. Contracts are changed in this report first.

10. Safety, security and privacy rules

10.1 Risk tiers

- **Tier 0, read-only:** runs immediately (find files, read a stored result, status).
- **Tier 1, reversible:** runs with a brief spoken heads-up (open app, site or folder, switch tabs, web search, run a registered test command, type text into an editor window Nova opened and verified).
- **Tier 2, impactful:** needs explicit confirmation (change or overwrite an existing file, type into a window Nova did not open, close an application).

- **Tier 3, forbidden in v1:** delete files, install software, anything with sudo, arbitrary shell commands, anything outside the allowed folders.

10.2 Folders

- Allowed: Downloads, Documents, Desktop, and the USB pen drive.
- Every path is resolved to its real location, following symlinks, and must sit inside an allowed folder. Parent-directory tricks are rejected.
- If the USB drive is missing, Nova says so and stops. It never falls back to another folder or project.
- Permission to read a place is not permission to change or delete there.

10.3 Execution rules

- No shell. Programs are started with a fixed argument list.
- The LLM supplies only a project name for tests, never the command.
- Every tool has a timeout (60 seconds by default), an output size limit, and its arguments are validated on the server.
- A request is limited to 5 steps. A failed step stops the request.

10.4 Network security

- The core is reachable on the home network, so anyone on that network could otherwise use the website. Rule: all endpoints that act, confirm, or push to the device need a shared access token, set once in config and entered once on the website.
- Read-only endpoints needed by the ESP32 and the phone stay simple.

10.5 Untrusted content

- Test logs and file contents fed back to the LLM are data, never instructions. In v1, web pages are not fed to the LLM at all (web search only opens the browser).

10.6 Privacy

- The microphone opens only after the wake word or push-to-talk, and the state is visible on the OLED and the website.
- Mic-off switch on the website, and 'stop listening' by voice.
- Audio is not stored. Logs keep text only.
- API keys live in environment variables or an ignored file, never in Git.

- Get consent before recording anyone else's voice.

11. Memory design (phase 7, planned now)

- **Where:** an Obsidian vault on the PC. The Python backend reads and writes the Markdown files. Obsidian is for you to browse and edit.
- **Structure:** a main brain as the root, holding who you are, preferences, standing rules and an index of sub-brains. Inside it, sub-brains like sub-repos. For now two: Ideas and Projects. More can be added later.
- **Ideas:** one short note per idea with the date, the cleaned text, a short title, tags, and the raw transcript (so speech errors can be corrected).
- **Projects:** one folder per project with an overview note (purpose, status, decisions, next actions).
- **Routing:** the assistant checks the root index to choose a sub-brain, or you say it directly ('open project X'). If unclear, it asks.
- **Retrieval:** read the index, then the sub-brain overview, then only the notes needed. Never load the whole vault.
- **Writes:** Nova drafts and you confirm anything important. Writes are limited to the vault with the same real-path check.
- **Separation:** live state (timer, to-dos, permissions, execution) stays in the backend. Short-term conversation context is separate from durable notes. No passwords or keys in notes.
- **Controls:** you can inspect, correct and delete anything Nova stored.

12. Working rules

- **Contract first:** update this report and the contracts, then code.
- **Git:** one repository, a branch per phase, small commits with clear messages, a tag at each phase exit. Secrets are ignored from day one.
- **Python:** type hints, small modules, logging instead of print, config instead of hard-coded values.
- **Firmware:** no blocking waits in the main loop, millis-based timing, shared data guarded by a mutex, no screen work from the network task.
- **Tests:** every tool and every safety rule gets a test before it is switched on. A phase cannot exit with failing tests.

- **Spikes:** time-boxed to about one day each, ending with a written decision.
- **Decision log:** each decision gets a line with date, choice and reason.
- **Change control:** bump the report version when a decision changes.
- **Coding agreement:** coding for a phase starts only when you say 'go ahead with code'.

13. Phases

Follow in order. Phases 0 to 6 are the core and fit the one-month goal if the work stays on plan. Phase 7 comes after the core works.

Phase 0: Foundations

- **Goal:** a clean repository and every decision that blocks later work answered.
- **Steps:** restructure the repo (server/nova, server/voice, firmware/nova_device, config, docs, tests); create config templates and .env.example; pin versions; reserve IPs for the laptop and ESP32; create the Picovoice key and check xAI access, credits and tool calling; confirm the Xfce app commands; note the USB mount path and the projects with their test commands; copy contracts into docs.
- **Exit:** a fresh clone starts the existing server and firmware from the README; open decisions 2 to 5 in section 16 are answered or defaulted.

Phase 1: Device states

- **Goal:** the ESP32 shows Nova's states when told.
- **Steps:** add the HTTP server task and shared state; draw the eight states and the text and step display; heartbeat and disconnected; time-to-live; token check; write a curl test script.
- **Exit:** each state looks right when sent by curl; unplugging the network shows disconnected within 15 seconds and recovers on its own; the eyes never freeze; the timer and to-dos still work.

Phase 2: Voice spikes

- **Goal:** choose components with measured numbers.
- **Steps:** test microphone levels; wake word 'Hey Nova' (20 attempts, plus an hour of idle listening to count false triggers); tune end-of-speech detection; compare tiny.en and base.en on 20 phrases for delay and mistakes; compare Piper and Kokoro (delay to first

audio, voice quality); test the LLM for tool calling, latency and cost; write results to docs.

- **Exit:** decisions logged; a clear yes or no on the 3-second target.

Phase 3: Text core

- **Goal:** typed chat drives tools and the OLED.
- **Steps:** core skeleton; provider interface and the Grok client; tool registry with schemas; mock tools; the agent loop (5 steps, verify, stop on failure); SQLite activity log; device client; website chat and activity panels; access token.
- **Exit:** typed requests run mock tools; the activity log shows each step; the OLED shows thinking, working, saved and error; a failing step stops the loop.

Phase 4: Voice loop

- **Goal:** talk to Nova and hear it answer.
- **Steps:** voice worker; wake word, recording, speech-to-text, send to core; text-to-speech playback; mic mute while speaking; push-to-talk; mic-off switch; acknowledgements; states pushed to the device; latency logging.
- **Exit:** a spoken conversation works with mock tools; median and slowest latency measured against 3 seconds; wake-word misses and false triggers recorded.

Phase 5: Real tools and safety

- **Goal:** Nova does real work, safely.
- **Steps:** path guard and USB check; app list; open site and search; tab switching; open folder and find files; registered test runs; type text with window verification; to-do and timer voice tools (if approved); confirmation flow on voice and website; forbidden list; safety test suite.
- **Exit:** every tool has tests; all safety tests pass; refused actions are explained clearly out loud.

Phase 6: Headline demo, evaluation and documentation

- **Goal:** prove it works and document it.
- **Steps:** build the headline demo including the missing-USB case; write and run the 30-command script (typed and a spoken subset); measure; fix failures; write the demo script, the safety document, the README and the final architecture update.

- **Exit:** at least 90% of commands pass; latency report written; the demo runs three times in a row.

Phase 7: Obsidian memory

- **Goal:** Nova remembers ideas and projects.
- **Steps:** create the vault; define the main brain, Ideas and Projects templates; build the guarded memory module; capture by voice; recall by voice; routing from the index; propose-and-confirm writes; memory controls; tests.
- **Exit:** you say an idea and a note appears with raw and cleaned text; later you ask and Nova finds it; nothing is ever written outside the vault; your own edits in Obsidian are seen by Nova.

Phase 8: Later ideas

- Interruption while speaking, screen awareness, interactive browser automation, more sub-brains, extra hardware (button, spare ESP32, USB microphone), local LLM option, other users.

14. Testing and evaluation

- **Scripted set of 30 commands:** opening apps and sites, search, folders and files, tab switching, typing text, running tests, blocked actions, ambiguous requests, failures such as a missing USB drive. Each has an expected outcome. Run typed to test the pipeline, and a subset spoken.
- **Measures:** success rate, delay from end of speech to first audio (median and slowest), wake-word misses out of 20, false triggers per hour, speech-to-text errors on 20 phrases.
- **Safety tests:** path tricks, symlink escapes, forbidden actions, missing USB, instruction-like text hidden in a test log, confirmation required, confirmation timeout.
- **Device tests:** every state, disconnect and recovery, eyes never freeze, timer survives.
- **Regression:** the full set runs before each phase exit. Results are saved in the docs.
- **Optional later:** a trial with 5 to 10 people, with consent before recording voices.

15. Risks and mitigations

- **Slow voice on a 2-core CPU:** measure in phase 2; smaller model, cloud speech-to-text, or acknowledgement-first.
- **Wake-word reliability or licence limits:** custom Porcupine model; fallbacks are openWakeWord and push-to-talk.
- **LLM access or cost (Grok):** verify in phase 0; swappable provider and a backup.
- **Echo:** headphones and mic mute while speaking.
- **Window control flakiness on X11:** verify the window before acting; stop and report on failure.
- **USB mount path changing:** configurable path and a clear 'drive not available' message.
- **IP changes:** reserved addresses or mDNS.
- **Flashing problems:** lower upload speed, better cable.
- **LAN exposure:** access token on every endpoint that can act.
- **Wrong tool choice by the LLM:** schemas, tiers, verification, and the step limit.
- **Scope creep against the one-month goal:** if time slips, cut typing text, tab switching, the Kokoro upgrade, and plans longer than 3 steps, in that order. Keep push-to-talk, safety tiers and the headline demo.

16. Open decisions (with defaults)

1. **Confirmation rule.** 'Both voice and website' could mean either channel may approve, or both must. Default: request on both, either approves, for tier 2 only.
2. **Grok API.** Check access, credits, pricing and tool calling. Default: verify in phase 0 and pick a backup provider.
3. **Wake word licence.** Check Picovoice's personal-use terms. Default: Porcupine, with openWakeWord as the fallback.
4. **USB drive and projects.** Mount path, project list and test commands for the registry.
5. **Xfce commands.** The real commands for the file manager, text editor, VS Code and browser.
6. **Access token.** Default: one shared token in config, entered once on the website.
7. **Voice control of to-dos and timer.** Default: include as tier 1 tools in phase 5.
8. **Activity log storage.** Default: SQLite.
9. **Python version.** Confirm in phase 0.
10. **Vault location and templates.** Decide at the start of phase 7.